Testing: JUnit 5, Mockito and Spring Slices

6 min read · Updated 28 Jul 2026 · Topic 62 of 65

Why it matters

Testing questions reveal how someone actually works. "Do you write tests?" is answered yes by everyone; "what do you mock, and what do you not?" separates people who have maintained a slow, brittle suite from people who have only added to one. The Spring-specific part — knowing that `@SpringBootTest` is usually the wrong annotation — is a strong, concrete signal.

The concept, simply

Three levels, each with a different cost and a different purpose:

- **Unit tests** — one class, collaborators mocked, no Spring at all. Milliseconds.
- **Slice tests** — a thin Spring context for one layer: `@WebMvcTest`, `@DataJpaTest`. Fast.
- **Integration tests** — the whole application with `@SpringBootTest`, ideally against a real database via Testcontainers. Slow, and the ones that catch what the others cannot.

The pyramid still holds: many unit tests, fewer slices, few end-to-end. The failure mode is inverting it — a suite of `@SpringBootTest` classes that takes fifteen minutes and gets skipped.

Code examples

JAVA Copy

```
// ✓ Unit test — no Spring, milliseconds. Possible ONLY because of
//   constructor injection (Chapter 11).
```

```
    @Mock OrderRepository repository;
    @Mock PaymentGateway gateway;
    @InjectMocks OrderService service;

    @Test
    void rejectsOrderWhenPaymentDeclined() {
        when(gateway.charge(any())).thenReturn(Result.declined("insufficie

        assertThatThrownBy(() -> service.place(anOrder()))
                .isInstanceOf(PaymentDeclinedException.class)
                .hasMessageContaining("insufficient");

        verify(repository, never()).save(any());     // ✓ assert what did
    }
}
```

Slice tests, which are what most people should be writing more of:

JAVA Copy

```
// ✓ Web slice — controllers, converters and validation only. No database,
//   no service beans; the service is mocked.
@WebMvcTest(OrderController.class)
class OrderControllerTest {

    @Autowired MockMvc mvc;
    @MockitoBean OrderService service;      // @MockBean before Boot 3.4

    @Test
    void returns404WhenMissing() throws Exception {
        when(service.find("x")).thenReturn(Optional.empty());

        mvc.perform(get("/api/v1/orders/x"))
           .andExpect(status().isNotFound())
           .andExpect(jsonPath("$.code").value("ORDER_NOT_FOUND"));
    }
}

// ✓ Persistence slice — JPA and an in-memory (or Testcontainers) database
```

```
// ... only. Each test is rolled back automatically.
@DataJpaTest
class OrderRepositoryTest {

    @Autowired OrderRepository repository;
    @Autowired TestEntityManager em;

    @Test
    void findsByReference() {
        em.persistAndFlush(anOrder("REF-1"));
        assertThat(repository.findByReference("REF-1")).isPresent();
    }
}
```

Integration tests against a real database — the modern default:

JAVA Copy

```
@SpringBootTest(webEnvironment = RANDOM_PORT)
@Testcontainers
class OrderIntegrationTest {

    @Container
    static PostgreSQLContainer<?> db = new PostgreSQLContainer<>("postgres

    @DynamicPropertySource
    static void props(DynamicPropertyRegistry registry) {
        registry.add("spring.datasource.url", db::getJdbcUrl);
        registry.add("spring.datasource.username", db::getUsername);
        registry.add("spring.datasource.password", db::getPassword);
    }

    @Autowired TestRestTemplate rest;

    @Test
    void createsAnOrder() { … }
}
// Why Testcontainers rather than H2: H2 is a DIFFERENT database. Its SQL
// dialect, constraint behaviour, isolation semantics and type handling al
```

Under the hood

Spring caches application contexts between test classes, keyed by configuration. Two classes with identical configuration share one context and the second starts instantly. Change anything — a different `@MockitoBean`, an extra property, a different set of active profiles — and Spring builds a *new* context. This is why a suite gets slow: not the number of tests, but the number of distinct configurations. Keeping test configuration uniform is the single biggest lever on suite runtime.

What to mock: things that are slow, non-deterministic or outside your control — HTTP clients, clocks, message brokers, third-party services. **What not to mock:** value objects, the classes you are testing, and — the common mistake — the database. Mocking a repository to return a fabricated entity tests your mocks, not your query, and every mapping and constraint bug slips straight through.

Assert behaviour, not implementation. A test that verifies six specific method calls in order breaks whenever anyone refactors, even when the behaviour is unchanged — so it costs more than it protects. Assert the outcome: what was returned, what was persisted, what was *not* called when it should not have been.

Common mistakes

- `@SpringBootTest` **for everything.** Loads the whole application for a test of one class.
- **Mocking the database.** Tests the mock, not the query — use `@DataJpaTest` or Testcontainers.
- **H2 standing in for Postgres.** A different database with different behaviour.
- **Over-verifying.** Asserting call sequences makes refactoring break tests that should not.
- **Tests depending on order or shared state.** Each must run alone and in any order.

- **Fragmenting test configuration.** Every distinct configuration is another context to build.
- **Field injection in production code.** Forces a Spring context just to construct the class under test.
- `Thread.sleep` **in async tests.** Slow and flaky — use Awaitility.

Interview lens

Q: "How do you structure tests in a Spring application?"

A: Mostly plain unit tests with Mockito and no Spring context — possible because constructor injection lets me build the class directly. Then slice tests for layer-specific behaviour: `@WebMvcTest` for controllers, serialisation and validation, `@DataJpaTest` for queries. Then a small number of `@SpringBootTest` integration tests against a real database via Testcontainers, covering the critical paths end to end.

Q: "What do you mock and what do you not?"

A: I mock what is slow, non-deterministic or outside my control — HTTP clients, clocks, brokers. I do not mock value objects, the class under test, or the database. Mocking a repository to return a fabricated entity means the test proves the mock behaves as configured; every mapping error, constraint violation and query mistake passes straight through. Those belong in a `@DataJpaTest` or an integration test.

Q: "Why is our test suite slow?"

A: Almost always too many distinct Spring contexts. Spring caches contexts across test classes keyed by configuration, so identical configurations are free — but any difference, an extra mock bean or a different property, forces a new one. Consolidating test configuration and replacing `@SpringBootTest` with slices where possible usually cuts runtime dramatically.

A: Testcontainers, for anything touching real SQL. H2 is a different database — dialect, constraint enforcement, type coercion and isolation behaviour all differ — so tests can pass against H2 and fail against Postgres, which is the worst possible outcome for a test suite. Testcontainers runs the actual database in Docker, so the test exercises what production runs.

Q: "How do you test asynchronous code?"
A: Never with `Thread.sleep` — that is slow when it passes and flaky when it fails. Awaitility polls a condition with a timeout, which is both faster and deterministic. Better still, inject the executor so the test can supply a same-thread one, making the async path synchronous and removing the timing question entirely.

Q: "What makes a good test?"
A: It asserts behaviour rather than implementation, so refactoring does not break it; it is independent of other tests and of order; it fails with a message that tells you what is wrong; and it is fast enough that nobody is tempted to skip it. Verifying exact call sequences fails the first criterion — those tests break constantly while protecting almost nothing.

Cheat-card

- **Unit (no Spring) → slice (`@WebMvcTest`, `@DataJpaTest`) → integration (`@SpringBootTest`).**
- Constructor injection is what makes plain unit tests possible.
- Mock: slow · non-deterministic · external. **Never mock the database.**
- **Testcontainers over H2** — H2 is a different database.
- Spring **caches contexts by configuration** → fewer distinct configs = faster suite.
- `@MockitoBean` (Boot 3.4+; was `@MockBean`) in slice tests.
- Assert **behaviour, not call sequences**. `verify(x, never())` is often the useful one.

QUICK CHECK — no marks, just for you.

1. Why should you not mock the database in a repository test?

- ◯ Because the test then proves only that the mock behaves as configured — every mapping error, constraint violation and query mistake passes straight through
- ◯ Because mocking is slower than a real database
- ◯ Because Spring forbids it
- ◯ Because Mockito cannot mock final classes

Check answer

2. Why prefer Testcontainers over H2 for tests that touch SQL?

- ◯ H2 cannot store dates
- ◯ H2 is a DIFFERENT database — dialect, constraint enforcement, type coercion and isolation behaviour all differ, so tests can pass against H2 and fail against Postgres
- ◯ H2 does not support transactions
- ◯ Testcontainers is faster to start

Check answer

3. Your Spring test suite is slow. What is the usual cause?

- ◯ Running tests in parallel
- ◯ Too many assertions per test

Too many DISTINCT Spring contexts — contexts are cached by configuration, so any difference such as an extra mock bean or a different property forces a new one

- ◯ Using JUnit 5 instead of JUnit 4

Check answer

‹ Previous
Spring Security and JWT

Next ›
SOLID in Practice

LEARN MORE

ARTICLES & DOCS

- Testing — Spring Boot reference
  docs.spring.io
- JUnit 5 user guide
  junit.org
- Testcontainers for Java
  java.testcontainers.org

VIDEOS



- Spring Boot Testing Tutorial - Part 3 | Testing REST APIs using MockMvc and Mockito
  Programming Techie



- Master Unit Testing Java Spring Boot REST API Application in One Shot | Full Course
  Think Constructive

Hand-picked references for this lesson.